AUTONOMOUS AI-AUGMENTED CI/CD PIPELINE WITH ADAPTIVE TRUST-TIERING AND CHAOS ENGINEERING

A PROJECT REPORT

Submitted by

Praveen Kumar C ([Roll No.])

Jaiprakash S ([Roll No.])

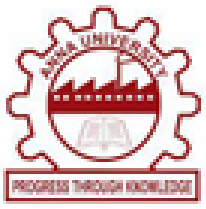
Dharshini T ([Roll No.])

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



DEPARTMENT OF INFORMATION TECHNOLOGY

MADRAS INSTITUTE OF TECHNOLOGY

ANNA UNIVERSITY

CHENNAI – 600 044

[MONTH YEAR]

ANNA UNIVERSITY : CHENNAI 60025

BONAFIDE CERTIFICATE

Certified that this project report "AUTONOMOUS AI-AUGMENTED CI/CD PIPELINE WITH ADAPTIVE TRUST-TIERING AND CHAOS ENGINEERING" is the bonafide work of Praveen Kumar C [Roll No.], Jaiprakash S [Roll No.], Dharshini T [Roll No.] who carried out the project work under my supervision.

SIGNATURE

[Name of the HOD]
HEAD OF THE DEPARTMENT
Department of Information Technology
Madras Institute of Technology,
Chennai – 44

SIGNATURE

Dr. P. Anandha Kumar
SUPERVISOR
Department of Information Technology
Madras Institute of Technology,
Chennai - 44

ACKNOWLEDGEMENT

We express our thankfulness to our project supervisor **Dr. P. Anandha Kumar**, Department of Information Technology, MIT Campus, for providing invaluable support and assistance with encouragement which aided us to complete this project.

We are thankful to the panel members **[Panel Member 1] and [Panel Member 2]**, Department of Information Technology, MIT Campus, for their invaluable feedback in reviews.

We express our gratitude and sincere thanks to our respected Dean of MIT Campus, **Dr. P. Jayashree**, for providing excellent computing facilities throughout the project.

PRAVEEN KUMAR C **[Roll No.]**

JAIPRAKASH S **[Roll No.]**

DHARSHINI T **[Roll No.]**

ABSTRACT

Failed CI/CD runs are still triaged by hand, although the correct response depends on the cause: a transient fault should be retried, a data fault escalated, a failing host quarantined and a real defect blocked. This project builds an AI-augmented release gate that classifies the distributed trace of a failed run into five operational classes and decides how much of the resulting action the agent may take alone.

Real HDFS traces from TraceBench are clock-aligned and split by scenario. An attention-pooled bidirectional LSTM reaches a macro-F1 of 0.7464 on 57 unseen scenarios and 0.6829 when REAL_BUG, whose perfect score is a workload confound, is excluded; an order-free baseline ties it. A trust engine grants four autonomy tiers from the Wilson lower bound of the agent's accuracy and demotes it at once when a real defect is merged. Supporting modules localise suspect spans, abstain outside the training domain, compare canary releases, test CVE reachability and inject faults.

On a live four-service system the agent earned full autonomy, lost it on a defect with no trace signature, and regained trust only through new evidence. The limits are stated: about 40% of healthy traces are flagged, compound faults are mostly missed, and the workflows and monitoring stack were not executed.

TABLE OF CONTENTS

CH. NO	TITLE	PAGE NO
	ABSTRACT	**iii**
	LIST OF TABLES	**vi**
	LIST OF FIGURES	**vii**
	LIST OF ABBREVIATIONS	**viii**
1	**INTRODUCTION**	**1**
	1.1 Continuous Delivery and the Triage Problem	1
	1.2 About the Project	1
	1.3 Contributions of the Project	2
	1.4 Problem Statement and Objectives	3
	1.5 Organization of the Project	4
2	**LITERATURE SURVEY**	**5**
	2.1 Trace-Based and Log-Based Failure Analysis	5
	2.2 Flaky Failures and Release Engineering	6
	2.3 Automation, Trust and Statistical Gating	7
	2.4 Root-Cause Localisation, Chaos Engineering and Dependency Risk	7
	2.5 Research Gap	8
3	**SYSTEM DESIGN**	**9**
	3.1 Overall Architecture of the Proposed System	9
	3.2 Dataset Description	10
	3.2.1 TraceBench HDFS	10
	3.2.2 Minishop live traces	11
	3.2.3 Supporting records	11
	3.3 Explanation of System Components	12
	3.3.1 Data foundation and clock alignment	12
	3.3.2 Triage model	12
	3.3.3 Trust engine	13
	3.3.4 Root-cause localisation	14
	3.3.5 Release gate and supporting modules	14
	3.3.6 Actuator and continuous-integration hook	15
	3.3.7 End-to-end workflow and integration	16
4	**DETAILED DESIGN**	**17**
	4.1 Data Foundation	17
	4.2 Triage Model	18
	4.3 Trust Engine	19
	4.4 Root-Cause Localisation, Drift Guard and Release Gate	20

	4.5 Service Interface, Metrics and Actuator	21
	4.6 Live System and Dashboard	22
5	**RESULTS AND DISCUSSION**	**23**
	5.1 System Requirements	23
	5.1.1 Software Requirements	23
	5.1.2 Hardware Requirements	23
	5.2 Experimental Setup and Evaluation Methodology	24
	5.3 Results for the Triage Model	24
	5.4 Results for the Trust Engine	27
	5.5 Results for Root-Cause Localisation	28
	5.6 Results for the Jaeger Adapter and the Drift Guard	29
	5.7 Results for the Security Gate	30
	5.8 Results for the Chaos Experiments	31
	5.9 Results for the Live System	32
	5.10 Verification and Engineering Findings	35
	5.11 Discussion and Threats to Validity	36
6	**CONCLUSION AND FUTURE ENHANCEMENT**	**38**
	6.1 Future Enhancements	39
	REFERENCES	**41**

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
2.1	Summary of the reviewed work and what each leaves open	8
3.1	Data used by the components of the system	10
3.2	Class taxonomy and the action each class maps to	11
4.1	Configuration of the triage model and its training	18
4.2	Promotion thresholds, retention floors and permitted actions per tier	19
4.3	Service interface of the triage and release-gate service	22
5.1	Software requirements of the system	23
5.2	Scenario-grouped split of the HDFS corpus	24
5.3	Macro-F1 and accuracy of the model and the baselines on the test set	25
5.4	Per-class results of the triage model on the test set	26
5.5	Top-k localisation rate against the proxy label	29
5.6	Accuracy of the model on three forms of the same 400 traces	30
5.7	Result of the vulnerability reachability analysis	31
5.8	Outcome of the chaos experiments (30 injections per row)	31
5.9	Agent accuracy by phase of the scripted live run	33
5.10	Trust-tier changes during the scripted live run	33
5.11	Defects exposed by measurement and the corrections made	36
5.12	Principal measurements and the context in which they apply	37

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
3.1	Overall architecture of the proposed system	9
3.2	Clock alignment of a parent and child operation recorded on different hosts	12
3.3	Architecture of the attention-pooled BiLSTM triage model	13
3.4	Trust tiers, promotion thresholds and demotion rules	14
3.5	Precedence of the release gate	15
3.6	The closed decision loop of the live system	16
5.1	Macro-F1 of the proposed model and the five baselines on the held-out test set	25
5.2	Confusion matrix of the triage model on the 10,513 test traces	26
5.3	Replay of the trust engine over the test stream, and the demotions after fault injection	28
5.4	Top-3 localisation rate of the proposed method and two baselines	29
5.5	Accuracy of the model on corpus records and on two forms of Jaeger export	30
5.6	Flagged rate and correct-class rate for each injected fault	32
5.7	Trust tier of the agent during the scripted live run	33
5.8	Live view of the dashboard at the end of the scripted run (393 requests served, three more than the 390 of Table 5.9)	34
5.9	Canary release of a slow-database version rolled back by the release gate	35
5.10	The HDFS model abstaining on a trace from an unrelated application	35

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
AST	Abstract Syntax Tree
BiLSTM	Bidirectional Long Short-Term Memory
CI/CD	Continuous Integration and Continuous Delivery
CPU	Central Processing Unit
CVE	Common Vulnerabilities and Exposures
HDFS	Hadoop Distributed File System
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LB	Lower Bound (Wilson)
LSTM	Long Short-Term Memory
OOV	Out-of-Vocabulary
OPA	Open Policy Agent
OTLP	OpenTelemetry Protocol
PR	Pull Request
RCA	Root-Cause Analysis
SRE	Site Reliability Engineering
SVM	Support Vector Machine
TF-IDF	Term Frequency - Inverse Document Frequency

CHAPTER 1

INTRODUCTION

1.1 Continuous Delivery and the Triage Problem

Modern software teams merge changes many times a day. Every change is built, tested and deployed by a continuous integration and delivery (CI/CD) pipeline [1]. When a pipeline run fails, a person has to decide what the failure means, and the correct response depends on the answer. A transient network or process fault should simply be retried. A data fault such as corruption is deterministic, so retrying it only burns build minutes and fails again. A failing host should be taken out of rotation. A genuine defect must block the merge and alert the developer. On a dashboard these four situations look alike: a red run. Flaky failures are common in practice [2], which makes the distinction expensive to make by hand and costly to get wrong in either direction.

Distributed tracing offers the evidence needed to make the distinction. A trace records one request as a tree of timed operations called spans, each tagged with its host and status [3], and open tools such as OpenTelemetry and Jaeger collect them [4] [5]. The difficulty is that reading a trace is a manual, expert task, and the clocks of the hosts that produced it do not agree.

A second question is rarely asked explicitly: how far may an automated agent be trusted to act on its own conclusion? Levels of automation have long been treated as a design variable rather than a switch [6]. An agent that has never been measured on a system should not merge code unattended, while one with a long, verified record may reasonably be allowed to retry a build without asking.

1.2 About the Project

This project builds an AI-augmented release gate for CI/CD pipelines. It classifies the trace of a failed run into one of five operational classes, **PASS**,

INFRA_FLAKE, **DATA_FAULT**, **ENV_FAULT** and **REAL_BUG**, maps the class to an action (allow, retry, escalate, quarantine, block), and decides how much of that action the agent may carry out alone from its measured track record. The work is organised into five cooperating parts.

- **Data foundation.** Real HDFS traces from the TraceBench data set are converted into clock-aligned, labelled operation sequences, and the split between training and test data is made by scenario so that no fault is seen on both sides.
- **Triage model.** An attention-pooled bidirectional LSTM (BiLSTM) classifies each trace and exposes the spans it attended to as evidence.
- **Trust engine.** Four autonomy tiers, T0 (observe) to T3 (act autonomously), are earned by a statistical test on the agent's own accuracy and lost immediately when it acts wrongly on a real defect.
- **Root-cause localisation.** Spans are ranked against a per-operation latency baseline and a postmortem is generated automatically.
- **Release gate.** A Jaeger adapter, a drift guard, a canary comparison, a vulnerability reachability check, a chaos harness and a dry-run actuator combine into one decision with an explicit precedence order.

The project also corrects an error in its own first phase. Phase 1 reported a validation accuracy of 88.18% that stayed frozen for three epochs. Investigation showed that each row of its input file was an event-count vector rather than a sequence, and that a rule using one column alone scores about 88.2% on the same set. This thesis therefore reports per-class results against explicit baselines and states the limits of every number.

1.3 Contributions of the Project

The contributions of this work, each measured in Chapter 5, are the following.

- A leakage-free sequence data pipeline for HDFS traces, in which clock alignment makes every one of the 81,777 traces orderable and the split is made by scenario.
- An attention-pooled BiLSTM triage model with five operational classes mapped to pipeline actions, reported per class against five baselines.
- A trust engine that grants autonomy from a statistical lower bound, withdraws it immediately on a missed defect, and counts only fresh evidence after a change.
- A drift guard whose second condition, an attested feature schema, exists because stock trace exports were measured to be misjudged four times in five.
- An integrated release gate, a closed-loop live system and an interactive dashboard that make the whole behaviour visible, including the case in which it fails.

1.4 Problem Statement and Objectives

The problem addressed is the automatic triage of failed CI/CD runs from trace evidence, together with a defensible answer to how much autonomy the triage agent should be given. The objectives are:

- to build a labelled, leakage-free sequence data set from real HDFS fault-injection traces;
- to train a classifier that beats order-free and count-based baselines on held-out scenarios;
- to gate autonomy on a statistical lower bound and to withdraw it when the agent is wrong;
- to localise suspect spans and to refuse to answer when a trace is outside the model's domain;

- to demonstrate the whole closed loop on a running system with injectable faults, and to report honestly where it fails.

1.5 Organization of the Project

This thesis is organized as follows:

- **Chapter 1: Introduction** presents the domain, the problem statement and the objectives.
- **Chapter 2: Literature Survey** reviews published work on trace-based failure analysis, flaky failures, automation and trust, root-cause localisation, chaos engineering and dependency risk, and derives the research gap.
- **Chapter 3: System Design** presents the architecture and the data sets and explains each component.
- **Chapter 4: Detailed Design** gives, for each component, its inputs, processing and one structured algorithm, together with the parameters used.
- **Chapter 5: Results and Discussion** lists the software and hardware requirements, reports the results of every component, and discusses limitations and threats to validity.
- **Chapter 6: Conclusion and Future Enhancement** summarises what was achieved, separates it from what was not, and proposes enhancements.

CHAPTER 2

LITERATURE SURVEY

This chapter reviews published research relevant to the five parts of the project. Section 2.1 covers trace-based and log-based failure analysis, Section 2.2 covers flaky failures and release engineering, Section 2.3 covers automation, trust and statistical gating, and Section 2.4 covers root-cause localisation, chaos engineering and dependency risk. Section 2.5 derives the research gap. Citation numbers follow the order in which the publications are first cited in the thesis.

2.1 Trace-Based and Log-Based Failure Analysis

Dapper, the tracing infrastructure of Google, showed that request-level traces can be collected continuously in production at low overhead and are useful for diagnosis [3]. X-Trace captured causal paths across layers of a networked system [7], and Pinpoint followed request paths through the components of an internet service and applied data mining to find the components associated with failures [8]. Together they established the causal call tree, rather than a flat log, as the unit of diagnosis, which is the representation used here.

Learning-based detection began mostly with logs. DeepLog models the sequence of log keys with an LSTM and flags a key that is not among the most likely successors [9]. LogRobust combines semantic vectors of log events with an attention-based BiLSTM to tolerate unstable log data [10]. An experience report compared supervised and unsupervised log-based detectors on public data sets and made the comparison reproducible [11]. For traces, Nedelkoski et al. use both the span sequence and its timing in a multimodal deep model [12], and Liu et al. detect anomalous microservice traces with service-level deep Bayesian networks [13]. These works detect anomalies; they do not assign an operational class that maps to a pipeline action, and none ties the decision to a measured right to act.

The TraceBench data set [14] fills part of this gap for data. It provides traces from the Hadoop distributed file system collected under injected network, process, data and system faults, and combinations of faults. Its scenario taxonomy is the source of the class labels used in this project. A study of microservice faults with a benchmark system and an empirical analysis of real faults [15] motivates treating fault type, not only fault presence, as the quantity to predict.

Attention mechanisms [16] and bidirectional recurrent networks [17] on top of LSTM cells [18] are the building blocks of the classifier. The attention weights are tempting as explanations, but it has been argued that they are not reliable explanations [19] and, in reply, that this depends on what is claimed [20]. This project follows the cautious reading: the attended spans are reported as a hint, never as proof.

Finally, distributed systems cannot assume that hosts agree on the time. Lamport's work on ordering events [21] is the classical reason to derive order from causal relations rather than timestamps, and the clock alignment of Section 3.3.1 applies the same idea to the recorded call tree.

2.2 Flaky Failures and Release Engineering

An empirical analysis of flaky tests found that asynchronous waiting, concurrency and test-order dependence are among their most common causes [2]. The cost of a flaky or misdiagnosed failure falls on the pipeline: retries consume time, and wrongly blocked or wrongly merged changes consume trust. Continuous delivery practice [1] treats the pipeline as an automated, repeatable release path, and site reliability engineering adds canary releases, error budgets and blameless postmortems as operating practices [22]. The project automates two of them: the canary comparison and the postmortem.

Machine learning inside such systems brings its own maintenance cost. Sculley et al. describe how a model's hidden dependencies and feedback loops create technical debt [23]. The closed loop of this project, in which the agent's own

decisions change its permissions, is such a feedback loop, and the live experiments of Chapter 5 show one concrete failure of it.

2.3 Automation, Trust and Statistical Gating

Parasuraman, Sheridan and Wickens describe automation as a range of levels across four functions: acquiring information, analysing it, selecting a decision and implementing an action [6]. The four trust tiers of this project are a deliberately coarse version of that idea. T0 stops after analysis, T1 recommends a decision, T2 implements actions that can be reversed and T3 implements all of them.

Moving between levels needs a decision rule. The Wilson score interval [24] is a classical interval for a binomial proportion, and a comparison of interval methods recommends it over the simple normal approximation for small samples [25]. Its lower bound is a conservative estimate of an accuracy measured on few trials: 17 correct out of 20 is 85%, but the lower bound is about 0.64. The project uses this bound as the promotion test. A separate caution comes from calibration research: the confidence of modern neural networks is often poorly calibrated [26], so the confidence value of the classifier is used only as a coarse floor and never as a probability of being correct.

2.4 Root-Cause Localisation, Chaos Engineering and Dependency Risk

Pinpoint [8] and later trace-analysis work localise faults by comparing failing requests with normal ones. The localiser of this project follows the same principle in its simplest form: each span is scored against the latency distribution of the same operation in healthy traces, and errors add a fixed bonus. It is evaluated against a stated proxy because no data set marks the faulty span.

Chaos engineering injects faults into a system on purpose, to test a hypothesis about its steady state [27]. The chaos harness of this project applies the idea to the agent itself: faults are injected into healthy traces and the question is whether the trust engine reacts correctly.

Open-source dependencies add risk that is easy to over-count. Ponta et al. argue that a vulnerable dependency matters when the vulnerable code can actually be reached from the application [28]. The security gate of this project uses a static form of that reasoning to decide which reported vulnerabilities may block a release.

2.5 Research Gap

The literature supplies detectors for logs and traces, data sets of faults, interval statistics and operating practices, but they are seldom combined. The gaps addressed here are: (i) a class taxonomy that maps directly to pipeline actions; (ii) an explicit, statistical and reversible rule for how much autonomy the classifier receives; (iii) a guard that abstains when a trace is outside the training domain; and (iv) an evaluation that reports weak numbers as plainly as strong ones. Table 2.1 summarises the works reviewed and what each leaves open. Chapter 3 describes a system that integrates the four.

Table 2.1: Summary of the reviewed work and what each leaves open

Work	Input	Output	Leaves open
Dapper, X-Trace, Pinpoint	Request traces and paths	Collection; path-based diagnosis	No operational class or action
DeepLog, LogRobust	Log-key sequences	Normal or anomalous	No fault type tied to an action
Nedelkoski et al.; Liu et al.	Traces with timing	Anomaly decision	No policy; no right to act
Luo et al.; Zhou et al.	Flaky tests; microservice faults	Empirical causes of failure	No automatic triage
Parasuraman et al.	Levels of automation	Conceptual model	No statistical rule for a level
Ponta et al.	Dependency code reachability	Reachable vulnerabilities	Not tied to a release decision
This project	Clock-aligned traces, canary, CVEs	Five classes, actions, tiers	Limits stated in Chapter 5

CHAPTER 3

SYSTEM DESIGN

3.1 Overall Architecture of the Proposed System

The system is organised in five layers, shown in Figure 3.1. Trace sources feed an adapter layer that normalises them and checks that the model is entitled to judge them. The decision layer holds the triage model, the root-cause localiser and the trust engine. The release-gate layer adds two independent signals, a canary comparison and a vulnerability check, and combines everything by a fixed precedence. The last layer executes the decision in dry-run mode, exports metrics and feeds the true outcome back to the trust engine.

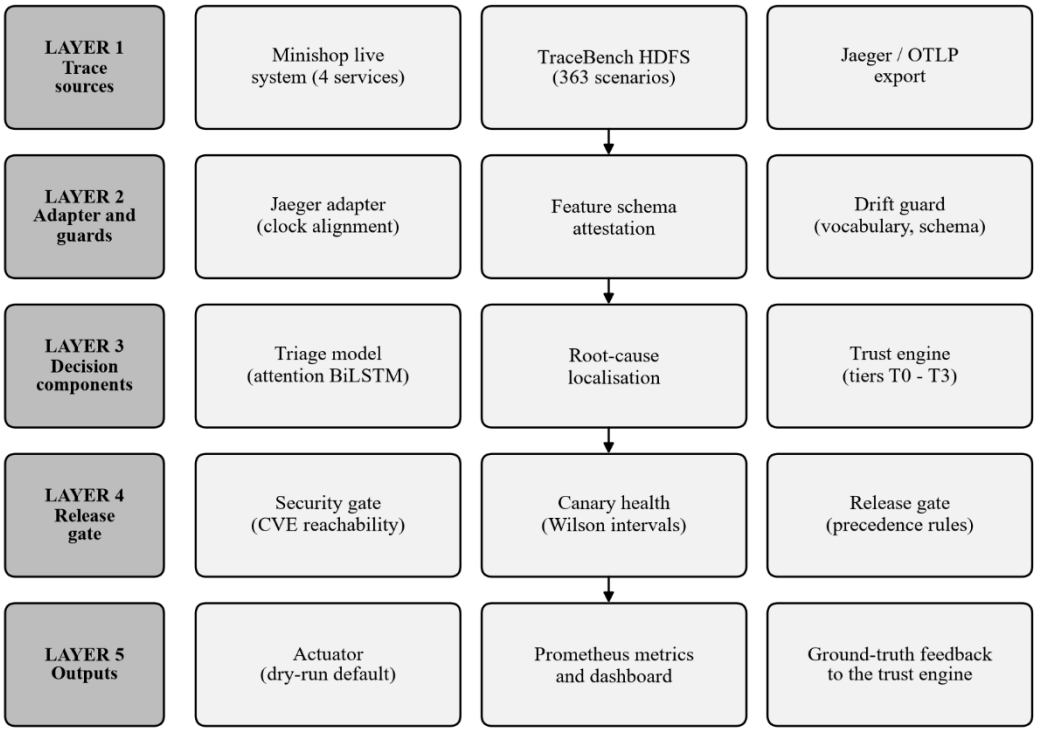


Figure 3.1: Overall architecture of the proposed system

Two design rules run through the layers. First, deciding and executing are separate: the gate and the policy only produce a decision, and a separate actuator, which contains no policy, carries it out. Second, the system prefers to abstain. A

trace outside the training domain, a canary with too little evidence and a confidence below 0.60 each route to a human instead of to an action.

3.2 Dataset Description

The system uses two sets of traces and two supporting records. They are described separately because no component merges them. Table 3.1 summarises them.

Table 3.1: Data used by the components of the system

Data	Used by	Content	Size and labels	Evaluation use
TraceBench HDFS v3	Triage, trust replay, RCA, chaos	Operation events and parent edges from HDFS under fault injection	363 scenario folders; 81,777 traces; 3.0 GB; class from folder family	Held-out scenarios; per-class metrics
Minishop live traces	Live loop, canary, dashboard	Jaeger-format traces of a 4-service application	37 experiments; 1,850 traces; exact labels from the injector	Pipeline demonstration only
HDFS Jaeger samples	Adapter, drift guard, gate	Five exported traces, four from HDFS and one from an unrelated application	One trace each	Qualitative
pip-audit report	Security gate	Vulnerabilities in the installed packages	27 findings	Reachability analysis

3.2.1 TraceBench HDFS

TraceBench [14] holds 363 scenario folders. Each folder is one experiment on a Hadoop distributed file system and contains the operation events of its traces and the edges between a parent operation and its children. Folder names encode the injected fault family. The mapping used here is given in Table 3.2. Scenarios that inject several faults at once (COMPOUND) are held out entirely and used only as a generalisation probe, 20,010 traces in all. The remaining 61,767 traces form the corpus, split by scenario into 40,967 training, 10,287 validation and 10,513 test traces.

Table 3.2: Class taxonomy and the action each class maps to

Class	TraceBench family	Meaning	Action	Min. tier
PASS	NM_CL, NM_DN	No failure signature	ALLOW_MERGE	T2
INFRA_FLAKE	AN_Net_*, AN_Proc_*	Transient network or process fault	AUTO_RETRY_PIPELINE (budget 2)	T2
DATA_FAULT	AN_Data_*	Corruption or loss; deterministic	ESCALATE_TO_INFRA	T1
ENV_FAULT	AN_Sys_*	Host-level failure	QUARANTINE_NODE	T3
REAL_BUG	COM_Sin_Bug	Genuine defect signature	BLOCK_MERGE_ALERT_DEV	T1

3.2.2 Minishop live traces

To run the closed loop without borrowing a production system, the project includes a small application, named minishop, with four services: a frontend, an order service, an inventory service and a database. Faults can be injected into it: a slow database and a flaky network (INFRA_FLAKE), data corruption (DATA_FAULT), a host failure (ENV_FAULT), a code bug that aborts the request (REAL_BUG), a novel bug that adds an unseen operation, and a silent bug that leaves the trace identical to a healthy one. The injector knows whether the fault took effect on each request, so every trace carries an exact label. A dependency-free tracer writes Jaeger JSON and OTLP/HTTP JSON. Thirty-seven labelled experiments produced 1,850 traces: 866 PASS, 318 INFRA_FLAKE, 122 DATA_FAULT, 300 ENV_FAULT and 244 REAL_BUG.

3.2.3 Supporting records

Five Jaeger exports serve the adapter and the drift guard: four real HDFS traces, one each of PASS, INFRA_FLAKE, DATA_FAULT and ENV_FAULT, and one trace from an unrelated routing application that the model must refuse to judge. The vulnerability report is the output of pip-audit on the project's own environment, 27 findings.

3.3 Explanation of System Components

3.3.1 Data foundation and clock alignment

Each event in TraceBench carries a start time on the clock of its own host, so sorting events by raw start time interleaves hosts incorrectly and drops the true root operation. The edge table records, for every child, the parent's clock reading when it spawned the child. The offset of a child's host relative to its parent's is therefore the difference of the two readings, and offsets are propagated down the call tree from the root, as Figure 3.2 illustrates. Ties are broken by depth so that a caller precedes its handler. All 81,777 traces align completely, and the aligned order recovers the true root operation. Each aligned event becomes a token of the form operation|status, with its duration, host role, agent type and depth kept as side features.

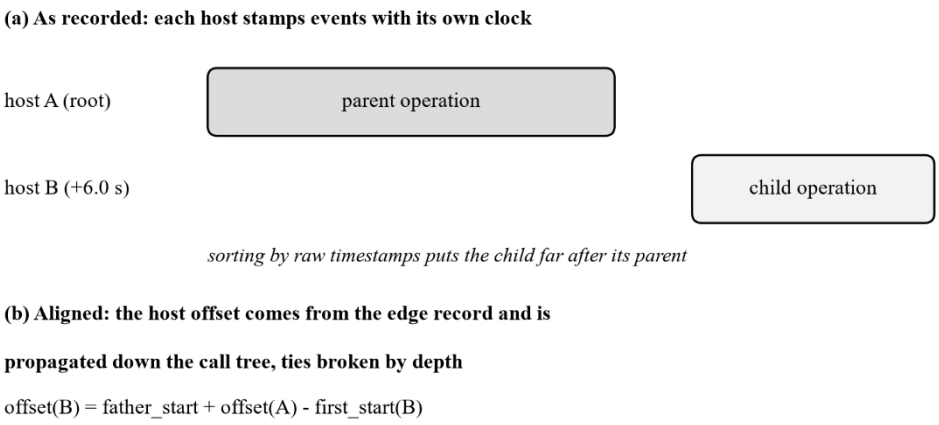


Figure 3.2: Clock alignment of a parent and child operation recorded on different hosts

3.3.2 Triage model

The classifier, shown in Figure 3.3, embeds each span token and concatenates small embeddings of the host role, the agent type and the depth, and a transformed duration. A one-layer bidirectional LSTM reads the sequence in both directions. Instead of keeping only the last hidden state, which is dominated by padding in a padded batch, the model pools all states with additive attention, so every span contributes in proportion to a learned weight. The attended vector, a max-pooled vector and nine trace-level scalar features feed a small dense head with five

outputs. The attention weights are returned with every prediction and identify the spans that drove it.

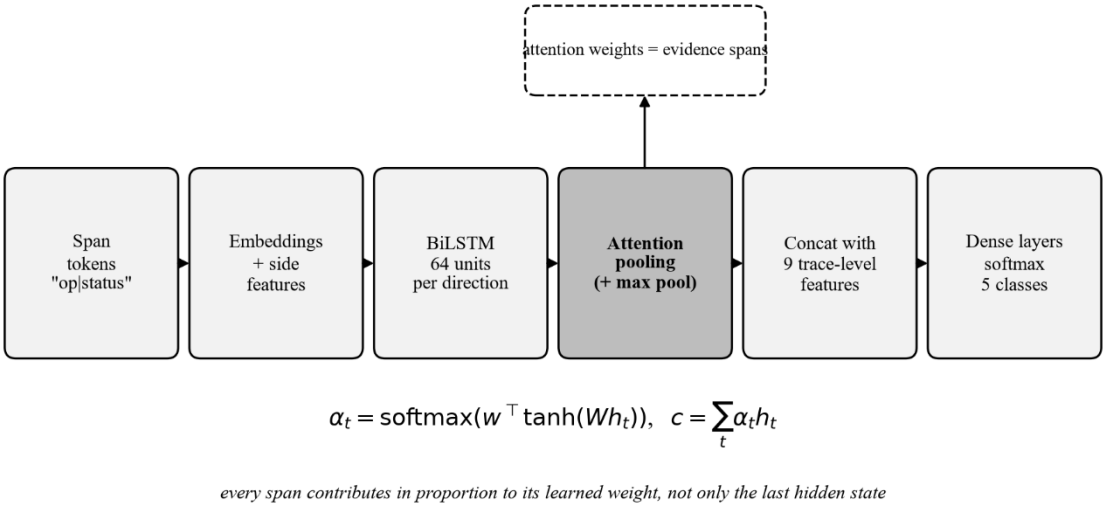


Figure 3.3: Architecture of the attention-pooled BiLSTM triage model

3.3.3 Trust engine

The trust engine (Figure 3.4) stores, for every agent, a tier and the history of its decisions. Promotion is decided by the Wilson lower bound of accuracy over the decisions whose true outcome is known, which prevents a short lucky streak from earning autonomy. Demotion has two triggers. The first is immediate: if the agent classified a real defect as something harmless and acted on it without a human, it drops one tier at once. The second is optional: if accuracy falls below a floor set ten points under the threshold that earned the current tier, the agent is demoted. Evidence is counted only from the last tier change onwards, so an agent cannot buy a tier straight back with the history that has just been proved insufficient.

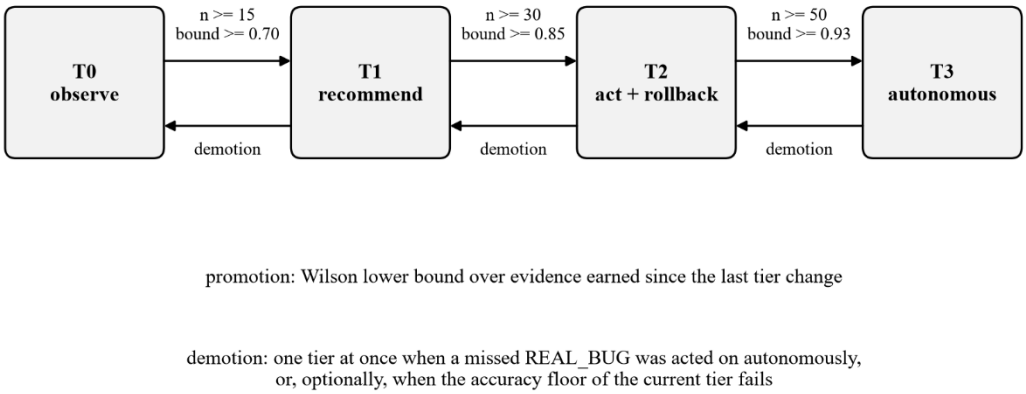


Figure 3.4: Trust tiers, promotion thresholds and demotion rules

3.3.4 Root-cause localisation

For every trace classed as a fault, the localiser ranks spans by suspicion. A baseline of mean and standard deviation of duration per operation is built from healthy traces. A span scores its standardised slowness (never negative), plus a fixed bonus of 2.0 if its own status is an error, plus a small bonus for sitting near the root. The top spans and a ready-to-paste postmortem in Markdown are returned.

3.3.5 Release gate and supporting modules

The release gate (Figure 3.5) combines four signals and chooses the most conservative. A security verdict of BLOCK wins over everything; a canary verdict of ROLLBACK comes next; a trace that fails the drift guard becomes a request for human review; otherwise the triage decision stands. The trust tier then decides whether the agent may execute the result or only recommend it. The drift guard checks two things: that the operation vocabulary of the trace is known to the model, and that the trace was produced with the same feature definitions the model was trained on. The canary module compares a new version's error rate and 95th-percentile latency with the baseline, using Wilson intervals so that few traces cannot cause a rollback. The security module decides which vulnerabilities the

running service can reach. The chaos harness injects six kinds of fault into healthy traces and a control with none.

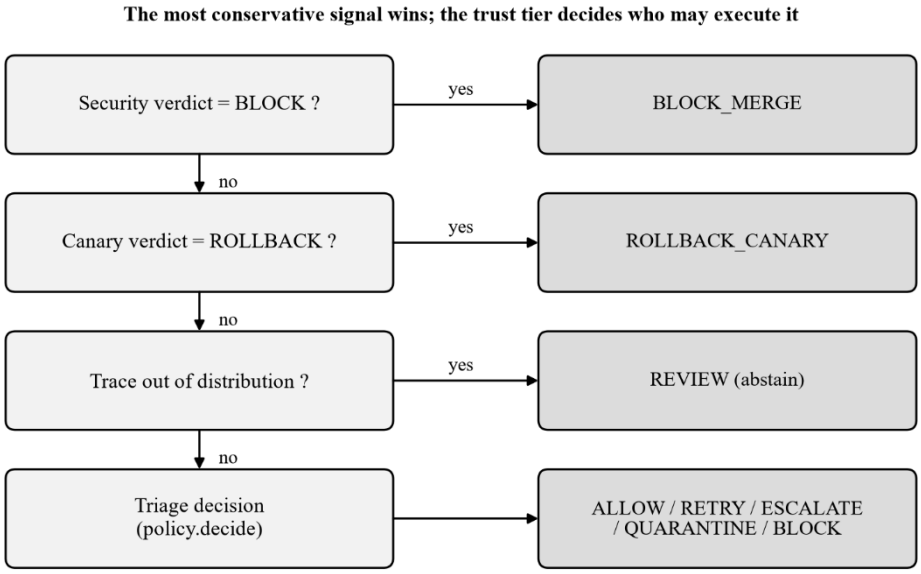


Figure 3.5: Precedence of the release gate

3.3.6 Actuator and continuous-integration hook

The actuator carries out a decision and contains no policy. By default it runs in dry-run mode and only records what it would do. Its real backend can set a commit status, post or update a single pull request comment marked so that it is not duplicated, rerun the failed jobs of a run, open an issue and run one configured command, such as cordoning a node or undoing a rollout. Commands are built as argument lists and never passed through a shell, and any node or deployment name that came from trace data is validated first, because traces are untrusted input. The workflow that calls the gate is triggered when the test workflow completes, so that it can rerun failed jobs, and it executes only code from the default branch; the artifacts of the pull request are treated as data and the pull request's code is never checked out in that workflow. Until the agent has earned a tier in deployment the check is advisory, since a T0 agent can only ask for a human.

3.3.7 End-to-end workflow and integration

Figure 3.6 shows the closed loop that the live system runs for every request. The request is served and traced, the adapter aligns the clocks and the drift guard checks the domain. The model, the policy and the gate produce a decision, which the actuator executes in dry-run mode if the tier allows. The true outcome is then reported back and the trust engine updates the tier, so the next request is governed by what the previous ones proved. Every component is called through one function, so the HTTP service, the command-line gate and the dashboard all use identical logic.

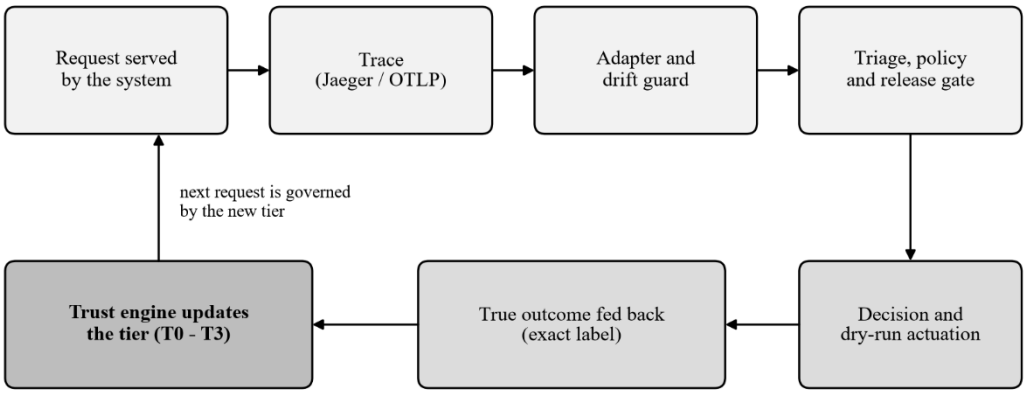


Figure 3.6: The closed decision loop of the live system

CHAPTER 4

DETAILED DESIGN

4.1 Data Foundation

The data foundation turns the raw TraceBench tables into labelled sequences. Algorithm 4.1 lists the steps for one trace. The offsets are recovered once per host from the first parent-child edge that reaches it and then reused down the tree. Operation names are normalised because several contain spaces, and the token stream is stored as a space-separated string; an unnormalised name would shift every later feature out of line, a defect that occurred once and is now covered by a test. The title of a task is deliberately not a model input. It records the client workload, and folder names encode the workload next to the fault, so feeding it would let the model recognise the scenario instead of the failure.

Algorithm 4.1: Clock alignment and token construction for one trace

```
Input : events of one trace (op, host, status, start, duration);
        edge records (father, child, father_start)
Output: ordered tokens, side features, trace-level scalars

 1. Build the call tree from the edge records. root <- event with no father.
 2. offset[host(root)] <- 0 ; queue <- [root]
 3. while queue is not empty:
      f <- pop(queue)
      for each child c of f:
         if host(c) has no offset yet:
            offset[host(c)] <- (father_start(c) + offset[host(f)])
                               - first_start(host(c))
            (father's clock reading, moved onto the common timeline, minus
             the first reading on the child's own host)
         push(queue, c)
 4. For every event e: t*(e) <- start(e) + offset[host(e)]
 5. Sort events by (t*(e), depth(e)); a caller precedes its handler on ties.
 6. token(e) <- normalise(op(e)) + "|" + status(e)
       normalise: spaces -> "_", "|" -> "/"
 7. side(e) <- (host role, agent type, depth, log(1 + duration))
 8. scalars <- log(1+n_events), log(1+n_thread_ids), log(1+n_hosts),
               log(1+n_edges), log(1+n_reports), log(1+latency),
               log(1+n_errors), n_errors / n_events, log(1+n_distinct_ops)
 9. Assign the class from the scenario-folder family (Table 3.2).
```

The split is made by scenario, never by trace. Each scenario is assigned once, keyed on its dominant non-PASS class, so that a scenario that serves both faulty and healthy requests cannot move classes between splits. The code asserts that no

trace and no scenario appears in two splits. This rule later exposed a real defect: an earlier per-class assignment silently emptied test classes in the small live corpus, which had produced an accuracy of 1.0 that meant nothing.

4.2 Triage Model

Algorithm 4.2 gives the inference procedure and Table 4.1 the configuration. The class weights in the loss are derived from class frequencies and softened by an exponent of 0.5. Full inverse-frequency weighting over-triggered ENV_FAULT, which has few examples, and merging ENV_FAULT into INFRA_FLAKE was rejected because the two map to different actions, quarantine and retry, and the distinction is the point of the project. Training uses the AdamW optimiser with a weight decay of 1e-4, a learning-rate scheduler that halves the rate when validation macro-F1 stops improving, label smoothing of 0.05, early stopping on validation macro-F1 and a fixed seed.

Table 4.1: Configuration of the triage model and its training

Item	Value	Item	Value
Token embedding	48	Dropout	0.3
Side embeddings (host, agent, depth, duration)	8 each	Batch size	256 (length-bucketed)
BiLSTM	1 layer, 64 per direction	Learning rate	0.003
Attention hidden size	64	Epochs (max) / patience	20 / 4
Max sequence length	256 tokens	Class-weight exponent	0.5
Vocabulary	87 tokens	Seed	42
Parameters	125,430	Training time (CPU)	1,427 s

Algorithm 4.2: Triage inference with attention pooling, drift check and policy

```
Input : tokens s_1..s_L (L <= 256), side features, scalars x
Output: class, confidence, evidence spans, action

 1. e_t <- [Emb(s_t) ; Emb_host ; Emb_agent ; Emb_depth ; W_d * dur_t]
          (48 + 3 x 8 + 8 = 80 dimensions)
 2. h_1..h_L <- BiLSTM(e_1..e_L)                  (64 units per direction,
                                                   packed: no padding steps)
 3. a_t <- w' tanh(W h_t)  for real positions t
    alpha <- softmax(a)                            (padding masked out)
 4. c <- sum_t alpha_t h_t ;   m <- max_t h_t      (128 dimensions each)
 5. z <- [c ; m ; ReLU(W_x x)]                     (128 + 128 + 32)
 6. p <- softmax(Dense(ReLU(Dense(z))))            (5 classes)
    class <- argmax p ;  conf <- max p
 7. evidence <- the 5 spans with the largest alpha_t
 8. oov <- share of tokens absent from the training vocabulary
 9. drift <- in_distribution if oov <= 0.5 and the feature schema is attested
10. action <- policy(class, conf, tier):
        conf < 0.60                    -> REVIEW (human)
        tier < min_tier(action)        -> recommend only
        otherwise                      -> execute
```

Batches are bucketed by length. Random batching pads every sequence to the longest member of its batch and wastes most of the compute on a CPU, and the model packs each batch so that the recurrent layer never processes a padding step.

4.3 Trust Engine

The thresholds rise with the tier because a wrong autonomous action at T3 costs more than one at T1. Table 4.2 lists them with the actions each tier may execute, using the minimum tiers of Table 3.2. Algorithm 4.3 is the update executed each time the true outcome of a decision becomes known.

Table 4.2: Promotion thresholds, retention floors and permitted actions per tier

Tier	Earned by (min. n, Wilson bound)	Retention floor	May execute
T0 observe	start	n/a	nothing; observation only
T1 recommend	15, 0.70	0.60	ESCALATE_TO_INFRA, BLOCK_MERGE_ALERT_DEV
T2 act with rollback	30, 0.85	0.75	adds ALLOW_MERGE, AUTO_RETRY_PIPELINE, ROLLBACK_CANARY
T3 autonomous	50, 0.93	0.83	adds QUARANTINE_NODE

Algorithm 4.3: Trust-tier update on a ground-truth outcome

```
Input : decision d (class, action, autonomous, tier) and its true label y
Output: updated tier and the reason for any change

 1. Store y with d ; correct <- (class(d) = y)
 2. if autonomous(d) and y = REAL_BUG and class(d) != REAL_BUG:
        tier <- max(tier - 1, 0) ; floor <- id(d) ; return "demoted"
 3. E <- ground-truthed decisions with id > floor, most recent 200
    n <- |E| ; k <- number of correct decisions in E
 4. LB(k, n) <- (p + z^2/2n - z*sqrt(p(1-p)/n + z^2/4n^2)) / (1 + z^2/n)
        with p = k/n and z = 1.96
 5. if tier < 3 and n >= n_min[tier] and LB(k, n) >= thr[tier]:
        tier <- tier + 1 ; floor <- id(d) ; return "promoted"
 6. if retention is enabled and tier >= 1:
        W <- last 50 decisions with id > floor ; n <- |W|
        if n >= 30 and LB(k, n) < thr[tier-1] - 0.10:
            tier <- tier - 1 ; floor <- id(d) ; return "demoted"
 7. return "hold"
        n_min = (15, 30, 50) and thr = (0.70, 0.85, 0.93) for T0, T1, T2
```

Step 2 is deliberately asymmetric with step 5. One wrong autonomous action on a real defect is enough to lose a tier, while promotion needs sustained evidence. Step 3 implements the rule that only evidence earned since the last tier change counts.

4.4 Root-Cause Localisation, Drift Guard and Release Gate

The localiser, given a trace and its class, scores every span as z + 2.0 x error + 0.25 / (1 + depth), where z is the standardised excess of the span's duration over the baseline mean of its operation and is floored at zero. Operations that never occur in healthy traces fall back to a global baseline. The result is the top five spans and a postmortem containing the ranked table and a one-paragraph summary.

Algorithm 4.4 is the release gate. The drift guard treats a trace as in-distribution only if at most half of its operations are unknown to the model and the exporter attests that the trace's trace-level features follow the training definitions. The second condition exists because of a measurement reported in Section 5.6: vocabulary alone does not protect the model from stock exports.

Algorithm 4.4: Release gate with drift guard and tier-governed execution

```
Input : trace (optional), security verdict, canary report (optional), tier
Output: recommended action, executed action, reasons

 1. if a trace is given:
        pred <- Algorithm 4.2 ; triage <- policy(pred.class, pred.conf, tier)
        drift <- vocabulary_ok(pred.oov) and schema_attested(trace)
 2. if security = BLOCK            : rec <- BLOCK_MERGE      ; rule <- security
 3. else if canary = ROLLBACK      : rec <- ROLLBACK_CANARY  ; rule <- canary
 4. else if drift = false          : rec <- REVIEW           ; rule <- ood-
abstain
 5. else if triage exists          : rec <- triage.action
 6. else                           : rec <- ALLOW_MERGE
 7. if rec = ALLOW_MERGE and security = REVIEW : rec <- REVIEW
    if rec = ALLOW_MERGE and canary   = HOLD   : rec <- REVIEW
 8. autonomous <- (rec != REVIEW) and (tier >= min_tier(rec))
 9. executed <- rec if autonomous or rec = REVIEW else REVIEW
10. log the decision; the actuator executes `executed` (dry-run by default)
```

The canary component computes error rates and 95th-percentile latency for both sets and a health score of 0.6 x error component + 0.4 x latency component. It answers ROLLBACK when the Wilson intervals of the error rates do not overlap or when the latency ratio exceeds 2.0, HOLD when fewer than 30 canary traces exist, and PROMOTE when the score is at least 0.80. The security component builds the import graph of the project with the standard abstract syntax tree, follows installed dependency requirements, and marks a vulnerability reachable only if its package is imported directly or transitively from the service's entry points.

4.5 Service Interface, Metrics and Actuator

One function, used by the HTTP service, the command-line gate and the dashboard, performs the whole evaluation, so that all three behave identically. Table 4.3 lists the service interface. The metrics endpoint renders four series in the Prometheus text format: decisions by class and action, the number of autonomous decisions, the trust tier of each agent and its accuracy over scored decisions. The actuator receives the result of the gate, and for every autonomous decision it sets a status, posts the comment and, for ENV_FAULT or a rollback, runs the configured command with a validated target. A retry budget of two limits repeated INFRA_FLAKE retries before the decision escalates to a human.

Table 4.3: Service interface of the triage and release-gate service

Method and path	Purpose
POST /api/v1/triage	Classify a trace given as tokens or spans; returns class, probabilities, action, evidence
POST /api/v1/triage/jaeger	Triage a Jaeger trace with the drift guard
POST /api/v1/triage/ground-truth	Report the true outcome; runs the trust engine and returns any tier change
POST /api/v1/gate	Full release gate: trace, canary sets, security verdict
POST /api/v1/postmortem	Ranked suspect spans and a Markdown postmortem
GET /api/v1/agent/scorecard	Measured accuracy and current tier of an agent
GET /api/v1/security/verdict	Stored reachability-aware vulnerability verdict
GET /metrics	Prometheus scrape endpoint

4.6 Live System and Dashboard

The minishop services sleep for real and retry on failure, and each request returns a trace and an exact label. The same Jaeger adapter that serves exported traces converts them, and the model is trained on the adapter's own output, so training and serving use identical features. A browser dashboard served by the project's API framework runs the closed loop continuously. It shows the pipeline stages, the latest trace tree, the trust tier over time and the decision feed, lets an operator inject any fault or run the scripted schedule, and has further views for canary releases, root-cause analysis, the security gate, the real-data model, chaos results and the metrics endpoint.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 System Requirements

5.1.1 Software Requirements

The software is written in Python and every reported number was produced with the versions in Table 5.1. The model is implemented in PyTorch [29], the baselines use scikit-learn [30], the service uses FastAPI, and Jaeger [5] runs in a container when a real trace back end is wanted. A Rego mirror of the policy for the Open Policy Agent [31] is included and checked against the Python policy by a test, but that test was skipped here because the opa executable was not installed. Metrics are exposed in the Prometheus text format [32].

Table 5.1: Software requirements of the system

Software	Version	Purpose
Python	3.13	Implementation language
PyTorch (CPU build)	2.13.0	Triage model, training
NumPy / pandas	2.5.1 / 3.0.5	Data handling
scikit-learn	1.9.0	Baselines, metrics
FastAPI / uvicorn / pydantic	0.141.1 / 0.52.1 / 2.13.4	Service and dashboard
NetworkX	3.6.1	Call-graph handling
pytest / pip-audit	9.1.1 / 2.10.1	Tests; vulnerability report
Jaeger all-in-one (Docker)	1.76.0	Real trace back end (optional)

5.1.2 Hardware Requirements

All training and measurement were done on one Windows 11 laptop without a GPU. Training the HDFS model took 1,427 seconds on the CPU. The corpus build over the 3.0 GB data set is resumable and takes a few minutes. The dashboard and the live loop run comfortably on the same machine alongside a Jaeger container.

5.2 Experimental Setup and Evaluation Methodology

The HDFS corpus is split by scenario folder into training, validation and test sets as shown in Table 5.2, so that a fault seen in training is never seen in testing. The test set therefore measures generalisation to unseen experiments. Macro-averaged F1 is reported because the classes are unbalanced, and a second figure that excludes REAL_BUG is reported for a reason given in Section 5.3. Every new claim is compared with at least one baseline that uses the same inputs or fewer. The five baselines are a majority-class rule (B1), a gradient-boosted model [33] on task-level scalars (B2), logistic regression on bag-of-operations counts, which is the representation of Phase 1 (B3), a support-vector machine on TF-IDF-weighted operation counts (B4), and gradient boosting on scalars plus operation counts (B5). B5 uses everything the BiLSTM uses except order.

Table 5.2: Scenario-grouped split of the HDFS corpus

Split	Scenarios	Traces	PASS	INFRA_FLAKE	DATA_FAULT	ENV_FAULT	REAL_BUG
Train	234	40,967	13,064	10,112	13,924	2,573	1,294
Validation	57	10,287	3,096	2,273	3,068	691	1,159
Test	57	10,513	2,964	2,319	3,068	832	1,330

5.3 Results for the Triage Model

Table 5.3 and Figure 5.1 compare the model with the baselines on the 10,513 test traces. The attention BiLSTM reaches a macro-F1 of 0.7464 and an accuracy of 0.7690, which is 0.043 above the strongest baseline, B5, and far above the bag-of-operations representation that Phase 1 used (0.5423). Restricted to the four classes other than REAL_BUG, however, the BiLSTM (0.6829) and B5 (0.6838) are tied, and the next two paragraphs explain why that is the figure to quote.

Table 5.3: Macro-F1 and accuracy of the model and the baselines on the test set

System	Macro-F1	Excl. REAL_BUG	Accuracy
Attention BiLSTM, class-weight exponent 0.5 (proposed)	0.7464	0.6829	0.7690
Attention BiLSTM, class-weight exponent 1.0	0.7297	0.6621	0.7314
B5 scalars + bag-of-operations, gradient boosting	0.7037	0.6838	0.7407
B2 task scalars, gradient boosting	0.6670	0.6588	0.7115
B4 bag-of-operations TF-IDF, SVM	0.5789	0.5598	0.5936
B3 bag-of-operations counts, logistic regression	0.5423	0.5174	0.5409
B1 majority class	0.0904	0.1129	0.2918

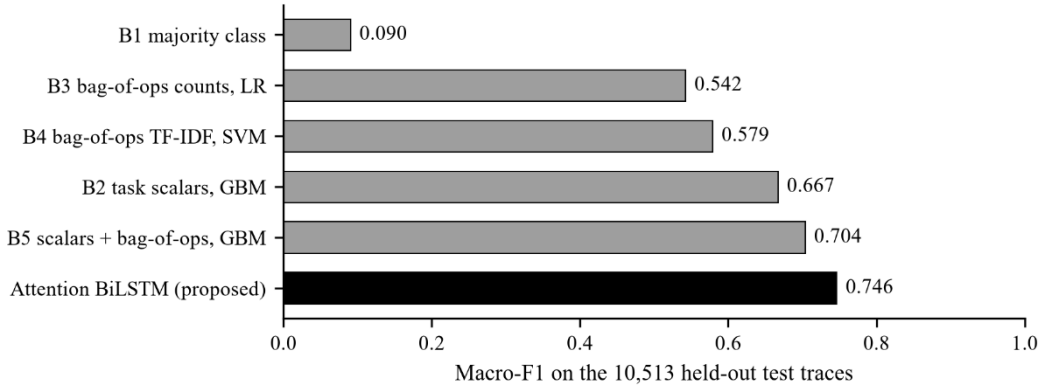


Figure 5.1: Macro-F1 of the proposed model and the five baselines on the held-out test set

Table 5.4 and Figure 5.2 give the per-class results. DATA_FAULT is found well (recall 0.913) and INFRA_FLAKE reasonably (recall 0.792). PASS has a recall of only 0.603, so about 40% of healthy traces are flagged as faults; this is the weakest operational number of the system and the chaos experiments of Section 5.8 reproduce it. ENV_FAULT is the weak class: precision 0.611 and recall 0.397. Softening the class weights from an exponent of 1.0 to 0.5 raised its F1 from 0.397 to 0.481 by trading recall (0.709 to 0.397) for precision (0.276 to 0.611), and which of the two matters more is a policy decision, since a false quarantine wastes capacity and a missed one leaves a bad node in rotation. Of the test traces, 139 (1.3%) were data faults predicted as INFRA_FLAKE and would have been retried, and no REAL_BUG was missed.

Table 5.4: Per-class results of the triage model on the test set

Class	Precision	Recall	F1	Support
PASS	0.7935	0.6029	0.6852	2,964
INFRA_FLAKE	0.7238	0.7922	0.7564	2,319
DATA_FAULT	0.7267	0.9126	0.8091	3,068
ENV_FAULT	0.6111	0.3966	0.4810	832
REAL_BUG	1.0000	1.0000	1.0000	1,330

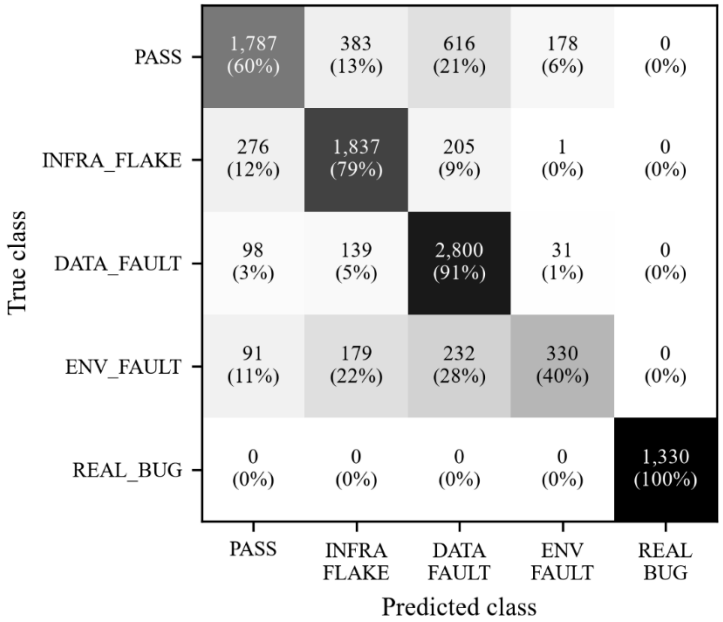


Figure 5.2: Confusion matrix of the triage model on the 10,513 test traces

REAL_BUG's score of 1.000 is a workload confound. The three REAL_BUG scenarios run a small metadata workload with a median of 7 spans on 2 hosts, whereas every other class has 88 to 134 spans on 9 to 12 hosts. Trace length alone separates them, so this is scenario recognition and not bug detection, and it inflates macro-F1 by about seven points. Only three such scenarios exist, one per split, so resplitting cannot repair it, and the figure excluding REAL_BUG is the honest headline.

Order carries little usable signal. The order-free B5 matches the BiLSTM on the other four classes, and an ablation that shuffles the tokens of every trace reached a validation macro-F1 of 0.7552 at epoch 3, against 0.7429 for the ordered model at the same epoch and 0.7476 at its best. The sequence pipeline is

nevertheless not wasted, because root-cause localisation needs the aligned call tree, but the classifier itself does not need order.

Compound faults mostly fail. The 20,010 held-out traces with several simultaneous faults, which the model never trained on, were classified PASS in 15,563 cases (77.8%). A multi-label output is the natural remedy and is listed as future work.

Attention is moderately concentrated, and two attempts to spread it were rejected. On 600 test traces of at least ten spans the mean weight of the top span was 0.54, 8.3% of traces put more than 90% of the weight on one span, and the normalised entropy was 0.48. An entropy bonus of 0.05 produced uniform attention (entropy 1.000) and a lower macro-F1 of 0.7408, and a hinge penalty gave 0.90 and 0.7439. Both made the evidence spans uninformative and were not adopted, so the evidence spans are reported as a hint only, consistent with the caution in [19].

5.4 Results for the Trust Engine

The trust engine was replayed over the 10,513 test decisions in order, with the true label supplied after each. Figure 5.3 shows the outcome. The agent reached T1 at decision 14, T2 at decision 906 and T3 at decision 1,130. The late promotions are expected: with an overall accuracy of 77%, a Wilson lower bound of 0.85 and then 0.93 is reached only through stretches of consistently correct decisions. A synthetic fault-injection segment was then appended, and the agent was demoted twice in succession and ended at T1.

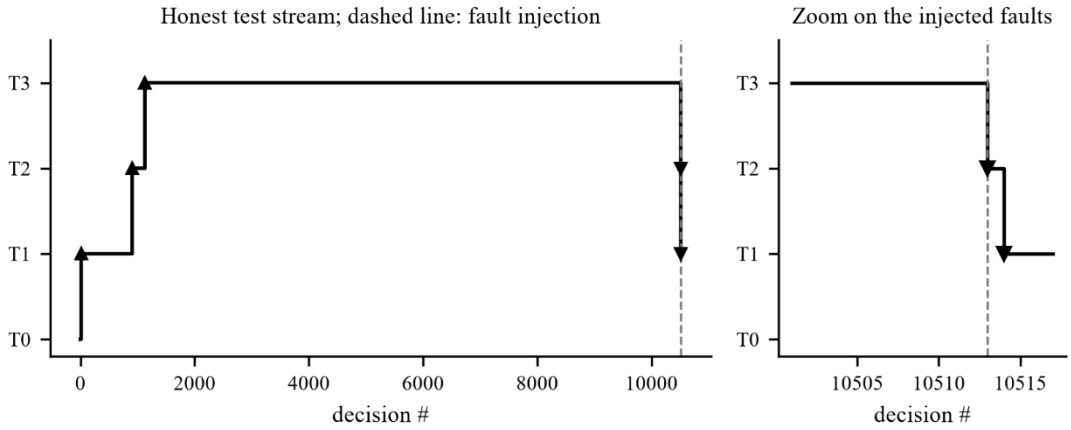


Figure 5.3: Replay of the trust engine over the test stream, and the demotions after fault injection

The live closed loop exposed a defect that unit tests had not. After a demotion, the next correct decision re-ran the promotion test over the same recent history that had just earned the tier, so the lower bound was high again and the tier was bought straight back: 53 tier changes in 50 decisions. The remedy, now in the engine, is that promotion and retention count only evidence gathered since the last tier change. The committed replay is unchanged by the fix (same promotions at the same decisions), while its injected-fault segment now ends at a stable tier instead of oscillating.

5.5 Results for Root-Cause Localisation

There is no data set that marks the faulty span, so localisation is evaluated against a proxy: the first span whose own status is an error. Table 5.5 and Figure 5.4 give the fraction of 10,411 fault traces for which that span is among the top k suspects, against random and slowest-span baselines. The method is clearly better than both for ENV_FAULT, INFRA_FLAKE and compound faults, with a top-3 rate of 94.6%, 73.4% and 91.0%. It is weak for DATA_FAULT: its top-3 rate of 47.6% is only a little above random (38.1%), because a corruption error is not slow and therefore gives the latency baseline nothing to find. These numbers measure agreement with the proxy and are not validated against independently labelled root causes.

Table 5.5: Top-k localisation rate against the proxy label

Fault class	Top-1	Top-3	Top-5	Random top-3	Slowest-span top-3
INFRA_FLAKE	0.593	0.734	0.995	0.323	0.506
ENV_FAULT	0.817	0.946	0.983	0.081	0.250
DATA_FAULT	0.262	0.476	0.778	0.381	0.150
COMPOUND	0.739	0.910	0.947	0.089	0.087

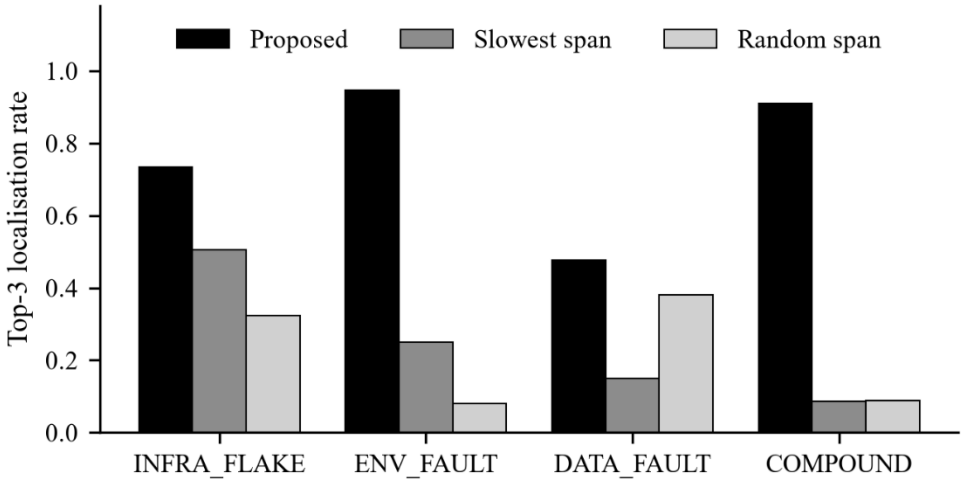


Figure 5.4: Top-3 localisation rate of the proposed method and two baselines

5.6 Results for the Jaeger Adapter and the Drift Guard

The model was trained on corpus records, not on Jaeger exports, so the cost of the export route was measured on 400 held-out traces (Table 5.6, Figure 5.5). On corpus records the accuracy is 0.74. On Jaeger exports that carry the training feature schema it is 0.63. On stock exports of the same traces it falls to 0.20, and 358 of the 400 stock traces (90%) come back as REAL_BUG, because the small thread and host counts of a stock export resemble the short REAL_BUG workload. This is the same confound as in Section 5.3 with a measured downstream cost, and it is the reason the drift guard requires both a known vocabulary and an attested feature schema. A trace from an unrelated routing application has an unknown-operation rate of 1.0 and is routed to a human.

Table 5.6: Accuracy of the model on three forms of the same 400 traces

Input form	Accuracy	Macro-F1	Agreement with corpus path
Corpus records (training format)	0.74	0.6913	1.00
Jaeger export with the training schema	0.63	0.5565	0.74
Stock Jaeger export	0.20	0.1277	0.20

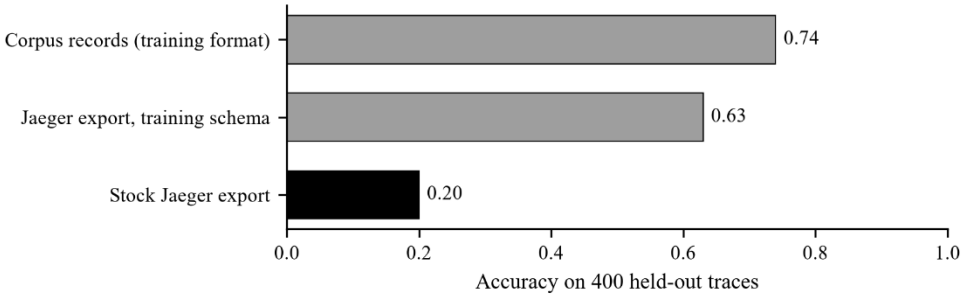


Figure 5.5: Accuracy of the model on corpus records and on two forms of Jaeger export

The Jaeger path was also checked against a real back end. The live traces were pushed over OTLP/HTTP to a Jaeger 1.76.0 container and appeared in its interface under the services frontend, orders, inventory and db, with seven-span traces. A poller then read twelve traces back from Jaeger and sent them to the agent API, and all twelve were judged in-distribution.

5.7 Results for the Security Gate

The pip-audit report of the project's environment lists 27 vulnerabilities: 12 in pip, 6 in tornado, 4 in jupyterlab, 3 in urllib3, and one each in jupyter-server and notebook. The reachability analysis found that none of the 27 packages is imported, directly or through the dependencies of the service, from the entry points of the running service (Table 5.7). The verdict is therefore ALLOW and the 27 findings do not block a release. A positive-control test shows that a flagged package that is imported is reported as reachable, and a simulated reachable vulnerability makes the gate answer BLOCK, which overrides any triage result. The analysis is static: a dynamic import or a package that loads an optional dependency on its own is invisible to it. The findings are still worth fixing by upgrading the affected packages.

Table 5.7: Result of the vulnerability reachability analysis

Measure	Value
Vulnerabilities flagged by pip-audit	27
Installed but not imported by the service	27
Reachable from the service entry points	0
Findings removed from the blocking decision	100%
Gate verdict	ALLOW (no reachable vulnerability)

5.8 Results for the Chaos Experiments

Six kinds of fault, built from signatures seen in real TraceBench fault traces, were injected into 30 healthy held-out traces each, and a control of 30 untouched traces was added. The agent started at T3 with retention demotion enabled. Table 5.8 and Figure 5.6 show the outcome. Every injected fault except the silent regression was flagged as a fault. The class was right for 97% of slow-datanode injections but only 50% of network disconnects, half of which were called DATA_FAULT. The control shows that 43% of healthy traces are flagged, the false-alarm floor that every fault row must beat. The silent regression has no trace signature by construction: none of its 30 traces was classified REAL_BUG, and two were acted on autonomously as unsafe merges. Those two immediately demoted the agent from T3 to T2 and from T2 to T1, and a retention failure on the data-node-kill experiments took it to T0: three demotions in all.

Table 5.8: Outcome of the chaos experiments (30 injections per row)

Injected fault	Should be	Flagged	Class correct	Unsafe actions
slow_datanode	INFRA_FLAKE	100%	97%	0
network_disconnect	INFRA_FLAKE	100%	50%	0
datanode_kill	INFRA_FLAKE	100%	83%	0
block_corruption	DATA_FAULT	100%	70%	0
readonly_datanode	ENV_FAULT	100%	80%	0
silent_regression	REAL_BUG	40%	0%	2
control (healthy)	PASS	43%	57%	0

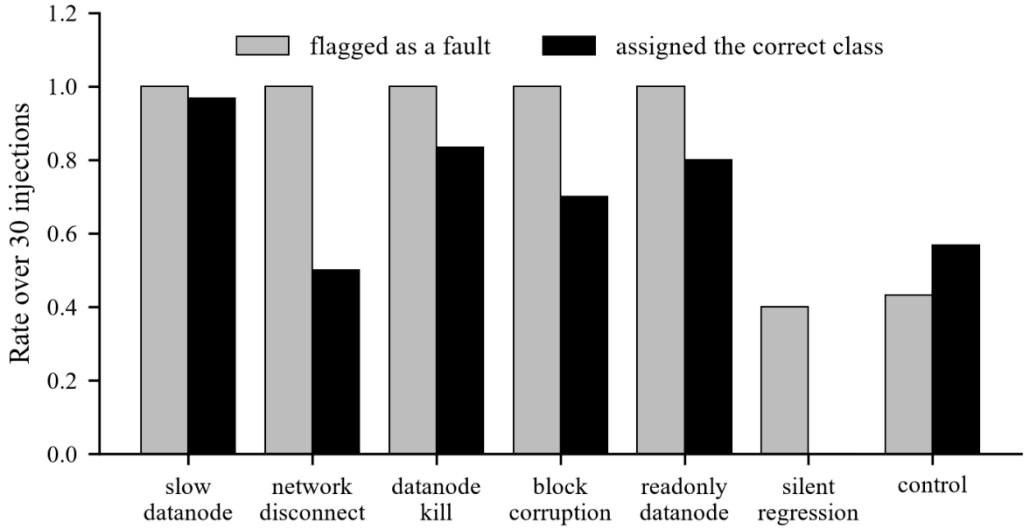


Figure 5.6: Flagged rate and correct-class rate for each injected fault

5.9 Results for the Live System

The live system was built to show the loop, not to benchmark the model. A model trained on the 1,850 minishop traces reaches a macro-F1 of 1.000 on the held-out scenarios, but so does the scalar gradient-boosting baseline (1.000), and the bag-of-operations baselines reach 0.975 and 0.884. The injected faults are simple, so this number says nothing about model quality, and the credible accuracy figure of the project remains the HDFS result of Section 5.3.

Figure 5.7 shows a run of the scripted schedule, which serves 390 requests in fifteen phases covering eight kinds of traffic. The agent decided correctly on 362 of 390 requests (92.8%) and made 2 unsafe autonomous actions. Table 5.9 breaks the result down by phase, and Table 5.10 lists the trust-tier changes. The agent starts at T0, is promoted at requests 15, 45 and 97, and reaches T3. Every fault type before the silent bug is classified correctly in every request. The silent bug, a real defect whose trace is identical to a healthy one, is classified correctly in 3 of 30 requests. Two of the first four requests of that phase were merged autonomously, and each demoted the agent, first from T3 to T2 and then from T2 to T1. After the phase ended, the accuracy retention check took it to T0 and, once healthy traffic had rebuilt 15 correct decisions, it was promoted to T1 again. The run therefore shows the behaviour the design intends: autonomy earned from

evidence, lost on a missed defect, and not regained without new evidence. Repeated runs differ by a few requests, because the services sleep for real, and gave between 92.0% and 92.9% correct decisions.

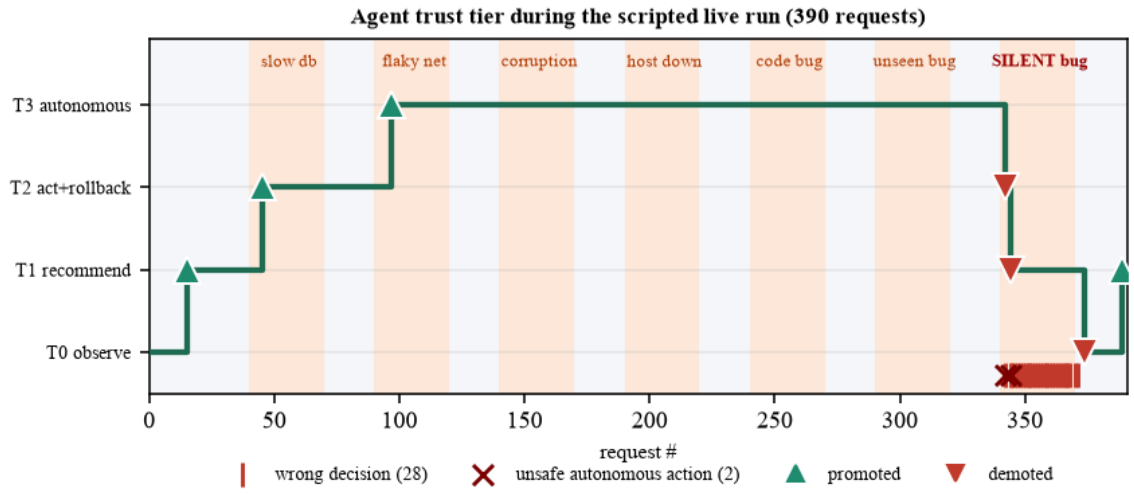


Figure 5.7: Trust tier of the agent during the scripted live run

Table 5.9: Agent accuracy by phase of the scripted live run

Phase	True class	Requests	Correct	Rate
Steady (healthy)	PASS	180	179	99%
Slow database	INFRA_FLAKE	30	30	100%
Flaky network	INFRA_FLAKE	30	30	100%
Data corruption	DATA_FAULT	30	30	100%
Host down	ENV_FAULT	30	30	100%
Code bug	REAL_BUG	30	30	100%
Unseen bug	REAL_BUG	30	30	100%
Silent bug	REAL_BUG	30	3	10%
All phases		390	362	92.8%

Table 5.10: Trust-tier changes during the scripted live run

Request	Change	Tiers	Reason
15	Promoted	T0 to T1	Wilson lower bound reached the threshold on fresh evidence
45	Promoted	T1 to T2	Wilson lower bound reached the threshold on fresh evidence
97	Promoted	T2 to T3	Wilson lower bound reached the threshold on fresh evidence
342	Demoted	T3 to T2	Missed real defect acted on autonomously
344	Demoted	T2 to T1	Missed real defect acted on autonomously
374	Demoted	T1 to T0	Retention floor failed (accuracy collapse)
389	Promoted	T0 to T1	Wilson lower bound reached the threshold on fresh evidence

The dashboard exposes the same loop interactively. Figure 5.8 shows its live view at the end of the scripted run, Figure 5.9 shows a canary release of a slow-database version being rolled back by the gate, and Figure 5.10 shows the HDFS model refusing to judge the trace of an unrelated application. A healthy new version is promoted by the canary check, a slow database raises the 95th-percentile latency 16.7 to 20.9 times in repeated runs, and a flaky network raised the error rate to 42.5% against 0%. When the agent is at a tier too low to act, the gate still reports its recommendation and states that it is advisory.

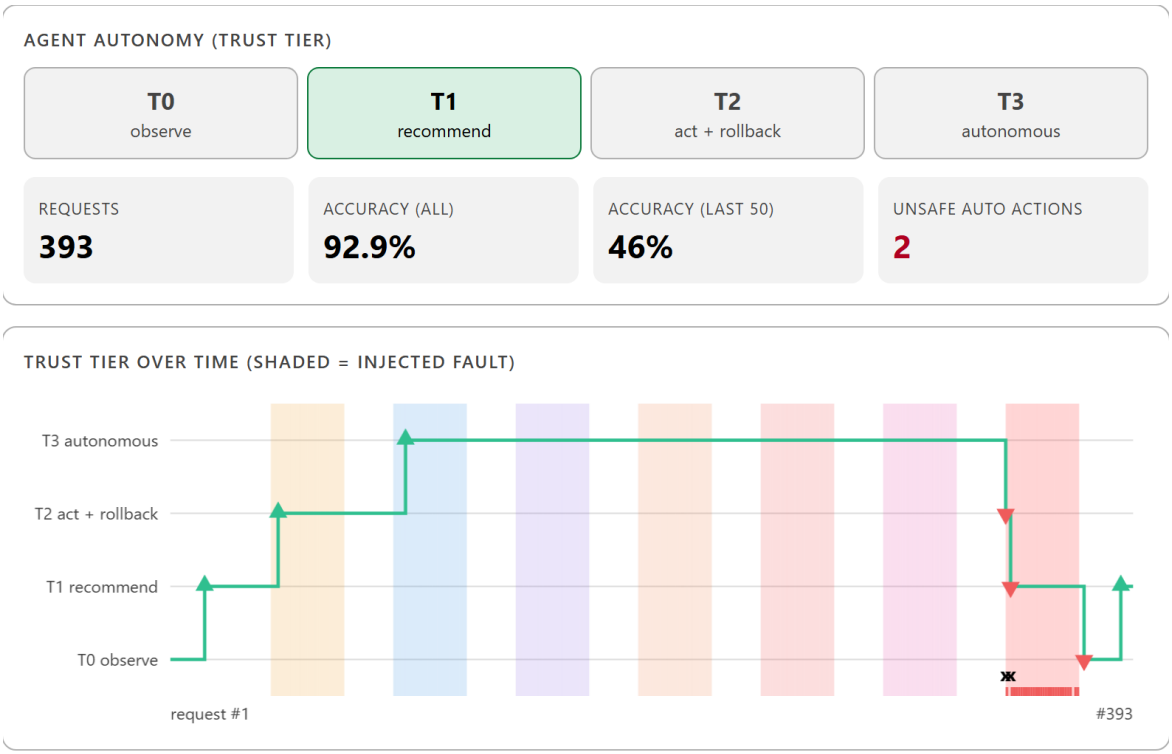


Figure 5.8: Live view of the dashboard at the end of the scripted run (393 requests served, three more than the 390 of Table 5.9)

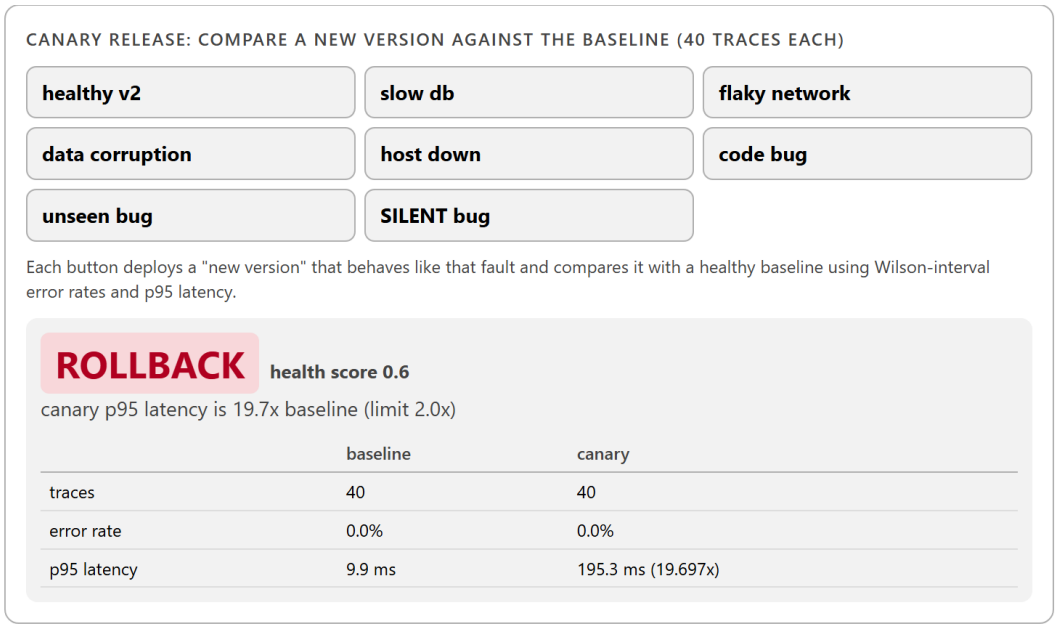


Figure 5.9: Canary release of a slow-database version rolled back by the release gate

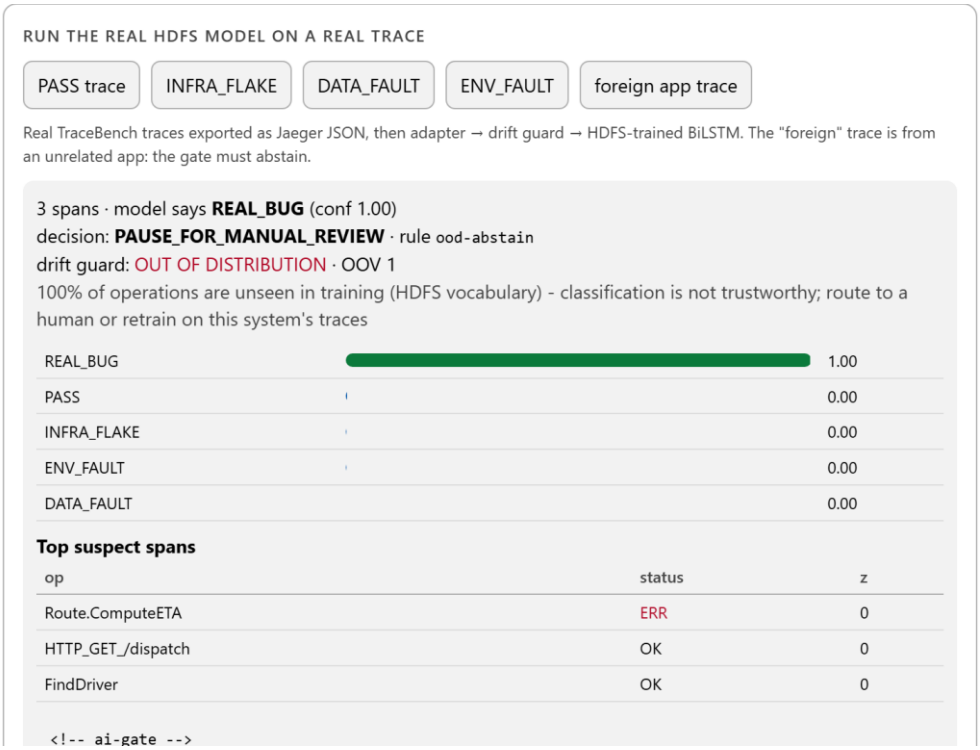


Figure 5.10: The HDFS model abstaining on a trace from an unrelated application

5.10 Verification and Engineering Findings

The behaviour described above is protected by an automated suite of 183 passing tests, with 12 skipped because the opa executable is not installed or because of one legacy marker. The tests cover the class taxonomy and policy, the leakage checks of the split, clock alignment, the trust engine, root-cause ranking, the

Jaeger adapter, the gate precedence, the security analysis with a positive control, the chaos harness, the actuator against a faked HTTP service, the live corpus and the dashboard endpoints. Several of the most useful results were not features but defects that measurement exposed. Table 5.11 lists them with the correction made.

Table 5.11: Defects exposed by measurement and the corrections made

Finding	Cause	Correction
Phase 1 accuracy of 88.18% frozen over three epochs	Rows were event-count vectors used as token ids; a one-column rule scores 88.2%	Input rebuilt from sequences; per-class metrics and baselines
Token features shifted out of line	Operation names containing spaces broke the space-separated token stream	Names normalised; covered by a test
Stock Jaeger exports judged with 20% accuracy	Missing trace-level features resemble the short REAL_BUG workload	Feature-schema condition added to the drift guard
Live test accuracy of 1.0 with classes missing	Per-class scenario assignment let PASS overwrite fault classes	Scenario assigned once by dominant fault class; HDFS split verified identical
53 tier changes in 50 decisions	A promotion re-used the evidence that had just failed to hold the tier	Only evidence since the last tier change counts
Some host agents fell into an unknown bucket	Agent-type lookup was case-sensitive	Lookup made case-insensitive
A demonstration reported an unknown-operation rate of 0.0	The text was fixed in code; the measured value was 0.47	The measured value is printed
Two seconds lost on every local call on Windows	The name localhost resolved through IPv6 first; 2,111 ms against 26 ms	All scripts use 127.0.0.1
Evidence spans lost their meaning	Attention regularisation spread weight almost uniformly	Both attempts rejected; spans reported as a hint

5.11 Discussion and Threats to Validity

Table 5.12 collects the principal measurements together with the context in which each applies. They belong to different tasks and are not combined into one score.

Table 5.12: Principal measurements and the context in which they apply

Measurement	Value	Context and caveat
HDFS test macro-F1	0.7464	57 held-out scenarios; 0.6829 excluding REAL_BUG, tied with B5
REAL_BUG recall	1.000	Workload confound: trace length separates the class
PASS recall	0.603	About 40% of healthy traces are flagged
Stock Jaeger export accuracy	0.20	90% called REAL_BUG; drift guard abstains
Root-cause top-3 (ENV / DATA)	0.946 / 0.476	Against a proxy label, not true root causes
Reachable vulnerabilities	0 of 27	Static analysis of imports only
Silent regression, class correct	0%	Two unsafe merges; trust fell T3 to T0
Live closed loop, correct decisions	92.8%	Toy system; 2 unsafe actions; T0 to T3 to T1
Live model macro-F1	1.000	Scalar baseline also 1.000: a pipeline demonstration
Automated tests	183 passed	12 skipped (no opa executable; one legacy marker)

The results are subject to the following threats to validity.

- **Domain.** The shipped model is valid only for HDFS traces in the training schema. The drift guard prevents a wrong answer outside it but cannot show that the model is right inside it.
- **Labels.** Class labels come from TraceBench's scenario taxonomy, so the model learns the scenarios that the taxonomy names. A single split and a single seed were evaluated; no confidence interval over seeds is given.
- **Confidence.** The confidence floor of 0.60 is an uncalibrated threshold on a softmax output [26].
- **Synthetic evidence.** The chaos faults copy signatures from real traces and the live faults are simple; both stress the pipeline but neither replaces the HDFS result.
- **Unexecuted infrastructure.** The two GitHub workflow files and the full container stack with Prometheus and Grafana were written and checked for structure but never run. Only Jaeger was run against a real server, and the actuator has only ever run in dry-run mode or against a faked HTTP service.

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENT

This thesis presented an AI-augmented release gate that classifies the trace of a failed CI/CD run, maps the class to a pipeline action and decides how much of that action the agent may carry out alone. A data foundation recovers the causal order of events on real HDFS traces by clock alignment and splits them by scenario. An attention-pooled BiLSTM reaches a macro-F1 of 0.7464 on 57 unseen scenarios, above a count-based representation of 0.5423 and above the strongest order-free baseline of 0.7037, with no real defect missed. A trust engine converts the agent's measured accuracy into four autonomy tiers by a Wilson lower bound and withdraws autonomy at once when a real defect is merged. Supporting modules localise suspect spans, abstain outside the training domain, compare canary releases, decide which vulnerabilities are reachable and inject faults to test the agent.

The honest reading of the results is narrower than the headline. The credible accuracy is the HDFS figure with REAL_BUG excluded, 0.6829, which ties an order-free baseline, so the recurrent sequence model does not improve classification over counts and scalars. REAL_BUG's perfect score reflects trace length, about 40% of healthy traces are flagged, ENV_FAULT is found in fewer than 40% of cases, and compound faults are mostly missed. Stock exports of the model's own traces are misjudged four times in five unless the drift guard abstains. The live system demonstrates the loop rather than the accuracy: the agent earned T3, lost it when a defect with no trace signature was merged, and recovered only through new evidence.

The achieved outcomes must be distinguished from the future objectives. Achieved are a documented architecture, a leakage-free data pipeline, a trained and baselined model, a statistically gated trust engine whose design was corrected

by a live experiment, a release gate with explicit precedence, a closed-loop live system with an interactive dashboard, a verified push and read-back through a real Jaeger server, and 183 passing automated tests. Not achieved are execution of the GitHub workflows, the Prometheus and Grafana stack, real chaos on infrastructure, and any evaluation on production traces.

6.1 Future Enhancements

The enhancements below are proposed. They are not existing capabilities, and they are ordered by the limitations identified above.

- **Train and evaluate on a real system.** Collect labelled traces from an application such as a microservice benchmark under real fault injection, with several independent scenarios per class, and report the result with the same per-class and baseline discipline.
- **Reduce false alarms.** Attack the 40% of healthy traces that are flagged, with a shape-matched control for REAL_BUG, a calibrated confidence and a cost-sensitive threshold.
- **Handle compound faults.** Replace the five-way output with a multi-label output so that simultaneous faults are not classified as healthy.
- **Make explanations trustworthy.** Try a much weaker attention penalty or multi-head pooling, and validate evidence spans against labelled root causes instead of a proxy.
- **Catch defects with no trace signature.** Add signals outside the trace, such as test results, business metrics or canary error budgets, because no trace-only model can see a silent regression.
- **Run the unexecuted infrastructure.** Execute the workflows on a repository, bring up the full container stack, and run chaos with real fault injectors against it; keep the AI gate advisory until the agent has earned T2 in deployment.

- **Persist trust in deployment.** Store trust state outside the process, feed the true outcome of every run back automatically, and alert when the agent is demoted.

REFERENCES

[1] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston, MA, USA: Addison-Wesley, 2010.

[2] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proc. 22nd ACM SIGSOFT Int. Symp. Found. Softw. Eng. (FSE)*, 2014, pp. 643–653.

[3] B. H. Sigelman et al., "Dapper, a large-scale distributed systems tracing infrastructure," Google Inc., Tech. Rep. dapper-2010-1, 2010.

[4] OpenTelemetry Authors, "OpenTelemetry documentation." [Online]. Available: https://opentelemetry.io

[5] Jaeger Authors, "Jaeger: Open source, end-to-end distributed tracing." [Online]. Available: https://www.jaegertracing.io

[6] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern. A, Syst. Humans*, vol. 30, no. 3, pp. 286–297, May 2000.

[7] R. Fonseca, G. Porter, R. H. Katz, S. Shenker, and I. Stoica, "X-Trace: A pervasive network tracing framework," in *Proc. 4th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2007, pp. 271–284.

[8] M. Y. Chen, E. Kiciman, E. Fratkin, A. Fox, and E. Brewer, "Pinpoint: Problem determination in large, dynamic Internet services," in *Proc. Int. Conf. Dependable Syst. Netw. (DSN)*, 2002, pp. 595–604.

[9] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2017, pp. 1285–1298.

[10] X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, 2019, pp. 807–817.

[11] S. He, J. Zhu, P. He, and M. R. Lyu, “Experience report: System log analysis for anomaly detection,” in *Proc. IEEE 27th Int. Symp. Softw. Rel. Eng. (ISSRE)*, 2016, pp. 207–218.

[12] S. Nedelkoski, J. Cardoso, and O. Kao, “Anomaly detection from system tracing data using multimodal deep learning,” in *Proc. IEEE 12th Int. Conf. Cloud Comput. (CLOUD)*, 2019, pp. 179–186.

[13] P. Liu et al., “Unsupervised detection of microservice trace anomalies through service-level deep Bayesian networks,” in *Proc. IEEE 31st Int. Symp. Softw. Rel. Eng. (ISSRE)*, 2020, pp. 48–58.

[14] J. Zhou, Z. Chen, J. Wang, Z. Zheng, and W. Dong, “TraceBench: An open data set for trace-oriented monitoring,” in *Proc. IEEE 6th Int. Conf. Cloud Comput. Technol. Sci. (CloudCom)*, 2014, pp. 519–526.

[15] X. Zhou et al., “Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study,” *IEEE Trans. Softw. Eng.*, vol. 47, no. 2, pp. 243–260, Feb. 2021.

[16] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” in *Proc. 3rd Int. Conf. Learn. Represent. (ICLR)*, 2015.

[17] M. Schuster and K. K. Paliwal, “Bidirectional recurrent neural networks,” *IEEE Trans. Signal Process.*, vol. 45, no. 11, pp. 2673–2681, Nov. 1997.

[18] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[19] S. Jain and B. C. Wallace, “Attention is not explanation,” in *Proc. NAACL-HLT*, 2019, pp. 3543–3556.

[20] S. Wiegreffe and Y. Pinter, “Attention is not not explanation,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 11–20.

[21] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Commun. ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978.

[22] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA, USA: O'Reilly Media, 2016.

[23] D. Sculley et al., "Hidden technical debt in machine learning systems," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 28, 2015, pp. 2503–2511.

[24] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[25] L. D. Brown, T. T. Cai, and A. DasGupta, "Interval estimation for a binomial proportion," *Statist. Sci.*, vol. 16, no. 2, pp. 101–133, 2001.

[26] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1321–1330.

[27] A. Basiri et al., "Chaos engineering," *IEEE Softw.*, vol. 33, no. 3, pp. 35–41, May/Jun. 2016.

[28] S. E. Ponta, H. Plate, and A. Sabetta, "Detection, assessment and mitigation of vulnerabilities in open-source dependencies," *Empir. Softw. Eng.*, vol. 25, no. 5, pp. 3175–3215, 2020.

[29] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[30] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[31] Open Policy Agent Authors, "Open Policy Agent documentation." [Online]. Available: https://www.openpolicyagent.org

[32] Prometheus Authors, "Prometheus: Monitoring system and time series database." [Online]. Available: https://prometheus.io

[33] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.